

关于体力值的服务器开发问题

一、体力值系统是否需要服务器回传？

需要服务器回传，原因如下：

- 1. 防作弊：防止玩家篡改本地时间或体力值数据（如通过修改手机时间无限恢复体力）¹³。
- 2. 跨设备同步：确保玩家在不同设备登录时体力值一致。
- 3. 数据安全：避免本地存储丢失导致进度重置。

二、整体解决方案

步骤1：配置抖音云服务器环境

1. 开通抖音云服务

- 登录抖音云控制台，创建按量计费的云环境（类似腾讯云开发，参考网页4的流程）

 无标题



云开发 部署 Cocos 游戏_w3cschool

<https://www.w3cschool.cn/cloudbasehosting/cloudbasehosting-q3n138be.html>

- 启用“云数据库”和“云函数”功能（用于存储体力值和进度数据）。
- 获取API密钥
 - 在控制台中生成 `AppID` 和 `SecretKey`，用于客户端与服务器通信时的身份验证。

步骤2：设计数据存储结构

在抖音云数据库中创建 `user_data` 表，包含以下字段：

字段名	类型	说明
user_id	string	用户唯一ID（抖音OpenID）
current_energy	int	当前体力值（初始10）
last_update_time	timestamp	最后一次体力恢复时间

game_progress	json	玩家关卡进度（可选）
---------------	------	------------

步骤3：客户端（Cocos）开发

1. 接入抖音云SDK

- 在Cocos项目中导入抖音云JavaScript SDK，初始化时填入 `AppID` 和 `SecretKey`。
- 示例代码（非程序员可交给开发人员）：

javascript



复制代码

```
const cloud = require('douyin-cloud-sdk');
cloud.init({ appId: 'YOUR_APP_ID', secretKey: 'YOUR_SECRET_KEY' });
```

体力值逻辑实现

- 本地显示：**参考网页3的体力值倒计时UI代码，但数据来源改为服务器接口3。
- 核心逻辑：**
 - 每次完成任务时，调用云函数 `reduceEnergy()` 扣除1点体力。
 - 体力不足时，调用 `getEnergyStatus()` 接口获取恢复倒计时或广告跳过选项1。

步骤4：服务器端（抖音云函数）开发

1. 云函数1：获取体力状态

- 输入：用户ID
- 逻辑：
 - 查询数据库中的 `current_energy` 和 `last_update_time`。
 - 计算离线期间恢复的体力值（公式： $\text{恢复量} = \text{floor}((\text{当前时间} - \text{最后更新时间}) / 300 \text{秒})$ ）。
 - 返回：当前体力值、下次恢复剩余时间、广告跳过次数。

云函数2：广告跳过冷却

- 输入：用户ID、广告类型
- 逻辑：
 - 验证广告播放完成回调（防止虚假请求）。
 - 直接增加5点体力值（不超过上限10）并更新数据库10。

步骤5：数据回传配置

1. 抖音广告回传（可选）

- 若需追踪广告带来的玩家行为（如通过广告进入游戏的用户体力消耗情况），在抖音广告后台配置“转化事件回传”（需技术人员协助）²¹⁰。
- 简化方案：使用第三方工具（如“多悦数据”）配置落地页参数回传，跳过代码开发¹⁰。

步骤6：测试与上线

1. 测试用例

- 断网恢复体力：离线2小时后上线，验证体力是否按5分钟/1点恢复。
- 广告跳过：连续观看3次广告，检查体力值是否限制在10点内³。

◦ 上线流程

- 将Cocos项目发布为抖音小游戏，并绑定抖音云环境。
- 在抖音开发者后台提交审核。

三、注意事项

1. 成本控制

- 抖音云按量计费，预估每日玩家数（如1万玩家，每日体力相关API调用约10万次，成本约50元/月）。

◦ 防作弊策略

- 服务器端校验时间戳，禁止客户端直接修改体力值¹³。

◦ 玩家引导

- 首次体力不足时弹出提示：“观看广告立即恢复5点体力，或等待自动恢复”。

四、参考资料

- Cocos体力值本地逻辑代码：[网页3]

<https://blog.csdn.net/K86338236/article/details/104773349>

- 抖音数据回传配置方法：[网页6]¹⁰

https://www.sohu.com/a/660170678_121675769

- 云开发部署指南（类比腾讯云）：[网页4]

- <https://www.w3cschool.cn/cloudbasehosting/cloudbasehosting-q3n138be.html>

体力值系统程序实现方案

一、技术栈说明

- 客户端：Cocos Creator 3.x (TypeScript)
- 服务端：抖音云开发 (云数据库 + 云函数)
- 数据流：客户端 → 抖音云函数 → 云数据库

二、核心代码实现

1. 抖音云数据库配置

在抖音云控制台创建数据库集合 `user_energy`，字段结构如下：

javascript

▼ 复制代码

```
// 集合名称: user_energy
{
  "_id": "自动生成ID",
  "user_id": "用户唯一标识 (抖音OpenID)",
  "current_energy": 10, // 当前体力值 (0-10)
  "last_update_time": 1672500000 // 最后更新时间 (Unix时间戳)
}
```

2. 服务端云函数开发

云函数1：获取体力状态 (getEnergyStatus)

typescript

▼ 复制代码

```
// 云函数代码 (TypeScript)
export async function main(event: any) {
  const db = cloud.database();
```

```

const userId = event.userId; // 从客户端传入

// 1. 查询用户数据
const res = await db.collection('user_energy').where({ user_id:
userId }).get();
let userData = res.data[0];

// 2. 若用户不存在，初始化数据
if (!userData) {
  userData = {
    user_id: userId,
    current_energy: 10,
    last_update_time: Date.now()
  };
  await db.collection('user_energy').add(userData);
}

// 3. 计算离线恢复体力
const now = Date.now();
const elapsedSeconds = Math.floor((now - userData.last_update_time) /
1000);
const recoveredEnergy = Math.floor(elapsedSeconds / 300); // 每5分钟恢
复1点

// 4. 更新体力值（不超过上限10）
const newEnergy = Math.min(userData.current_energy + recoveredEnergy,
10);
await db.collection('user_energy')
  .doc(userData._id)
  .update({
    current_energy: newEnergy,
    last_update_time: now
  });

// 5. 返回客户端数据
return {
  currentEnergy: newEnergy,
  nextRecoveryTime: 300 - (elapsedSeconds % 300) // 剩余恢复时间（秒）
};
}

```

云函数2：通过广告恢复体力（recoverEnergyByAd）

typescript

[复制代码](#)

```
export async function main(event: any) {
  const db = cloud.database();
  const userId = event.userId;

  // 1. 验证广告是否真实完成（抖音广告SDK回调）
  const adValid = await cloud.callFunction('verifyAdCompletion', {
    adId: event.adId
  });
  if (!adValid) throw new Error('广告验证失败');

  // 2. 更新体力值（+5点，不超过上限10）
  const res = await db.collection('user_energy')
    .where({ user_id: userId })
    .get();
  const userData = res.data[0];

  const newEnergy = Math.min(userData.current_energy + 5, 10);
  await db.collection('user_energy')
    .doc(userData._id)
    .update({
      current_energy: newEnergy,
      last_update_time: Date.now()
    });

  return { success: true, currentEnergy: newEnergy };
}
```

三、客户端代码实现（Cocos）

1. 初始化抖音云SDK

typescript

[复制代码](#)

```
// 文件: CloudManager.ts
import { douyinCloud } from 'douyin-cloud-sdk';

export class CloudManager {
  static init() {
```

```

douyinCloud.init({
  appId: 'YOUR_APP_ID', // 从抖音云控制台获取
  secretKey: 'YOUR_SECRET_KEY'
});
}

// 调用云函数通用方法
static async callFunction(name: string, data: any) {
  return douyinCloud.callFunction({
    name,
    data
  });
}
}

```

2. 体力值管理模块

typescript



复制代码

```

// 文件: EnergyManager.ts
export class EnergyManager {
  private currentEnergy: number = 0;
  private nextRecoveryTime: number = 0;

  // 初始化时获取体力状态
  async init() {
    const userId = douyin.getUserInfo().openId; // 获取抖音用户ID
    const res = await CloudManager.callFunction('getEnergyStatus', {
      userId
    });
    this.currentEnergy = res.currentEnergy;
    this.nextRecoveryTime = res.nextRecoveryTime;
    this.scheduleRecovery();
  }

  // 定时更新恢复时间
  private scheduleRecovery() {
    setInterval(() => {
      if (this.nextRecoveryTime > 0) {
        this.nextRecoveryTime--;
        if (this.nextRecoveryTime === 0) {
          this.currentEnergy = Math.min(this.currentEnergy + 1, 10);
        }
      }
    });
  }
}

```

```

    }, 1000);
  }

  // 消耗体力
  async consumeEnergy() {
    if (this.currentEnergy <= 0) return false;

    const userId = douyin.getUserInfo().openId;
    await CloudManager.callFunction('consumeEnergy', { userId });
    this.currentEnergy--;
    return true;
  }

  // 通过广告恢复体力
  async recoverByAd(adId: string) {
    const res = await CloudManager.callFunction('recoverEnergyByAd', {
      userId: douyin.getUserInfo().openId,
      adId
    });
    this.currentEnergy = res.currentEnergy;
  }
}

```

四、关键配置与调试

1. 抖音广告SDK集成

在 `project.json` 中添加抖音广告插件配置：

`project.json`

复制代码

```

{
  "plugins": {
    "douyin-ad": {
      "version": "1.0.0",
      "provider": "douyin-official"
    }
  }
}

```

2. 安全规则配置

在抖音云控制台设置数据库安全规则：

json



复制代码

```
{
  "read": "auth.openid == doc.user_id",
  "write": "auth.openid == doc.user_id"
}
```

五、测试用例

1. 正常恢复流程测试

typescript



复制代码

```
// 测试代码示例
const energyManager = new EnergyManager();
await energyManager.init();
console.log(energyManager.currentEnergy); // 应输出10

// 模拟等待5分钟
setTimeout(async () => {
  await energyManager.init();
  console.log(energyManager.currentEnergy); // 应输出10（已达上限）
}, 300000);
```

2. 广告恢复测试

typescript



复制代码

```
// 模拟广告播放回调
douyinAd.onAdComplete((adId) => {
```

```
energyManager.recoverByAd(adId);
console.log(energyManager.currentEnergy); // 例如从5变为10
}):
```

六、部署流程

1. 上传云函数

- 将 `getEnergyStatus` 和 `recoverEnergyByAd` 函数代码打包为ZIP，上传至抖音云函数管理界面。
- 客户端构建
 - 在Cocos Creator中选择抖音小游戏平台，构建发布。
- 联调测试
 - 使用抖音开发者工具进行真机调试，确保广告回调与云函数正常通信。

七、常见问题解决

问题现象	解决方案
广告播放后体力未恢复	检查广告ID是否回传到云函数
离线恢复时间计算错误	确保服务器时间使用UTC时间戳
数据库读写权限不足	检查安全规则是否配置正确

该方案可直接交付开发团队实施，如需进一步简化流程，建议使用抖音云提供的「数据同步模板」快速生成基础代码（参考抖音云文档第4章）。